Министерство образования Республики Беларусь

Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Программирование мобильных приложений»

ОТЧЁТ
к лабораторной работе №4
на тему
«ОНЛАЙН-ИГРА МОРСКОЙ БОЙ»
БГУИР 6-05-0612-02 23

Выполнил студент группы 353503
СЕБЕЛЕВ Дмитрий Юрьевич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
БУРАКОВ Дмитрий Игоревич

_____
(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Постановка задачи 3
2 Выполнение работы 4
2.1 Архитектура приложения 4
2.2 Профиль пользователя 6
2.3 Создание лобби и расстановка кораблей 6
2.4 Игровой процесс 7
2.5 Навигация и выход из игры 8
Вывод 9
Приложение А (обязательное) Листинг программного кода 10

1 ПОСТАНОВКА ЗАДАЧИ

Целью лабораторной работы являлась разработка мобильного приложения «Морской бой» для операционной системы *Android*. Приложение предназначено для игры двух пользователей по сети с использованием удаленного хранения состояния партии.

В соответствии с заданием к лабораторной работе необходимо было реализовать следующие требования:

– два игрока должны иметь возможность участвовать в игре с разных устройств;

– *Firebase Firestore*, *RealtimeDB* или аналогичный сервис должен использоваться в качестве бэкенда;

– должна быть реализована аутентификация пользователя с использованием любого провайдера или пары email-пароль;

– пользователь 1 создает игру и делится сгенерированным ID игры с противником;

– пользователь 2 присоединяется к игре по полученному ID;

– устройство пользователя 1 выступает в роли сервера и отвечает за обновление состояния игры;

– устройство пользователя 2 подписывается на изменения состояния в базе данных и обновляет интерфейс;

– должна быть реализована страница статистики пользователя с предыдущими играми;

– должна быть реализована страница профиля пользователя с возможностью изменения никнейма и выбора аватара;

– для хранения данных должно использоваться удаленное хранилище Firebase;

– должна быть предусмотрена корректная обработка отказа в доступе к разрешениям Android.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Архитектура приложения

Приложение разработано на языке *Kotlin* с использованием *Jetpack Compose* для построения пользовательского интерфейса. Для хранения данных и синхронизации состояния игры используется *Firebase Firestore*, а для авторизации пользователей – *Firebase Authentication*.

Основные части приложения:

– *SeaBattleApp* – основной компонент навигации между экранами;

– *AppViewModel* – управление авторизацией и профилем игрока;

– *GameViewModel* – управление состоянием лобби, расстановкой кораблей и ходами;

– *GameRepository* – работа с коллекциями Firestore;

– *GameEngine* – логика обработки выстрелов и определения победителя;

– экраны *ProfileScreen*, *LobbyScreen* и *BattleScreen* – пользовательский интерфейс основных частей приложения.

На рисунках 2.1, 2.2 и 2.3 представлены основные диаграммы проекта.

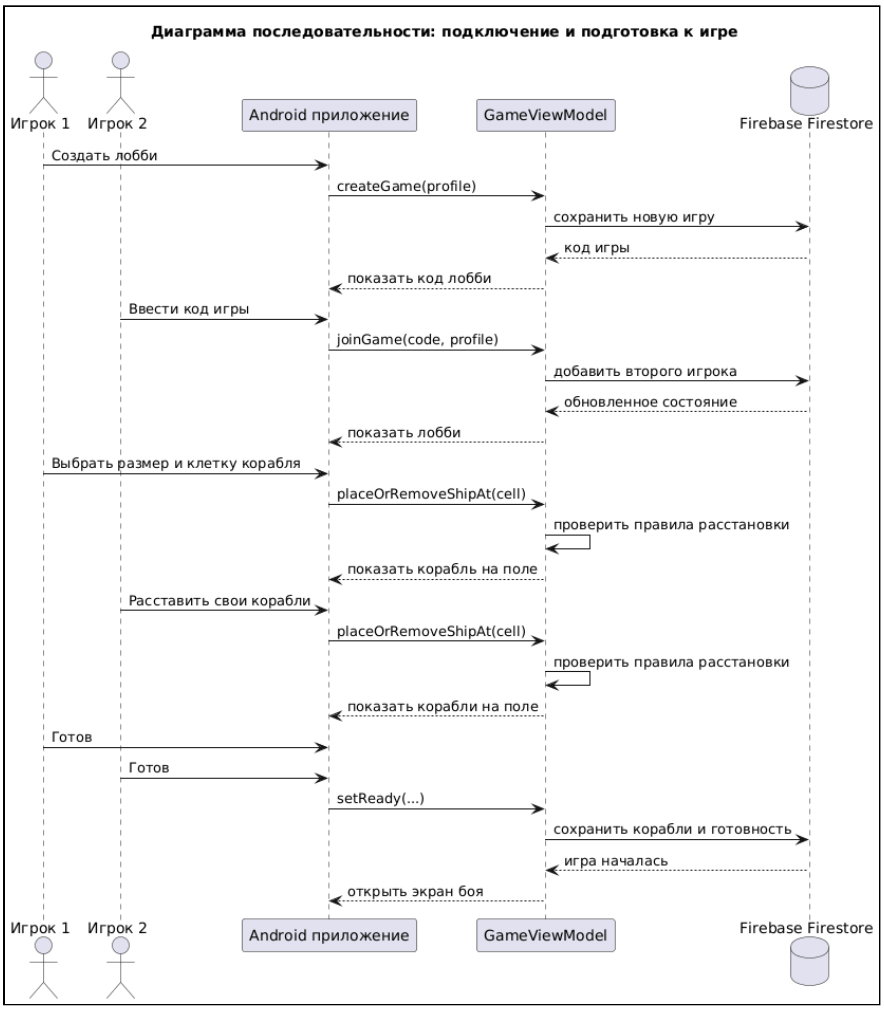

Рисунок 2.1 – Диаграмма последовательности подключения к игре

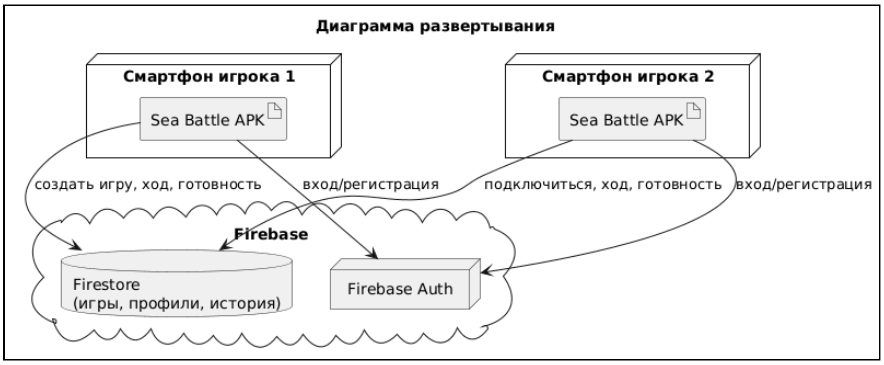

Рисунок 2.2 – Диаграмма развертывания приложения

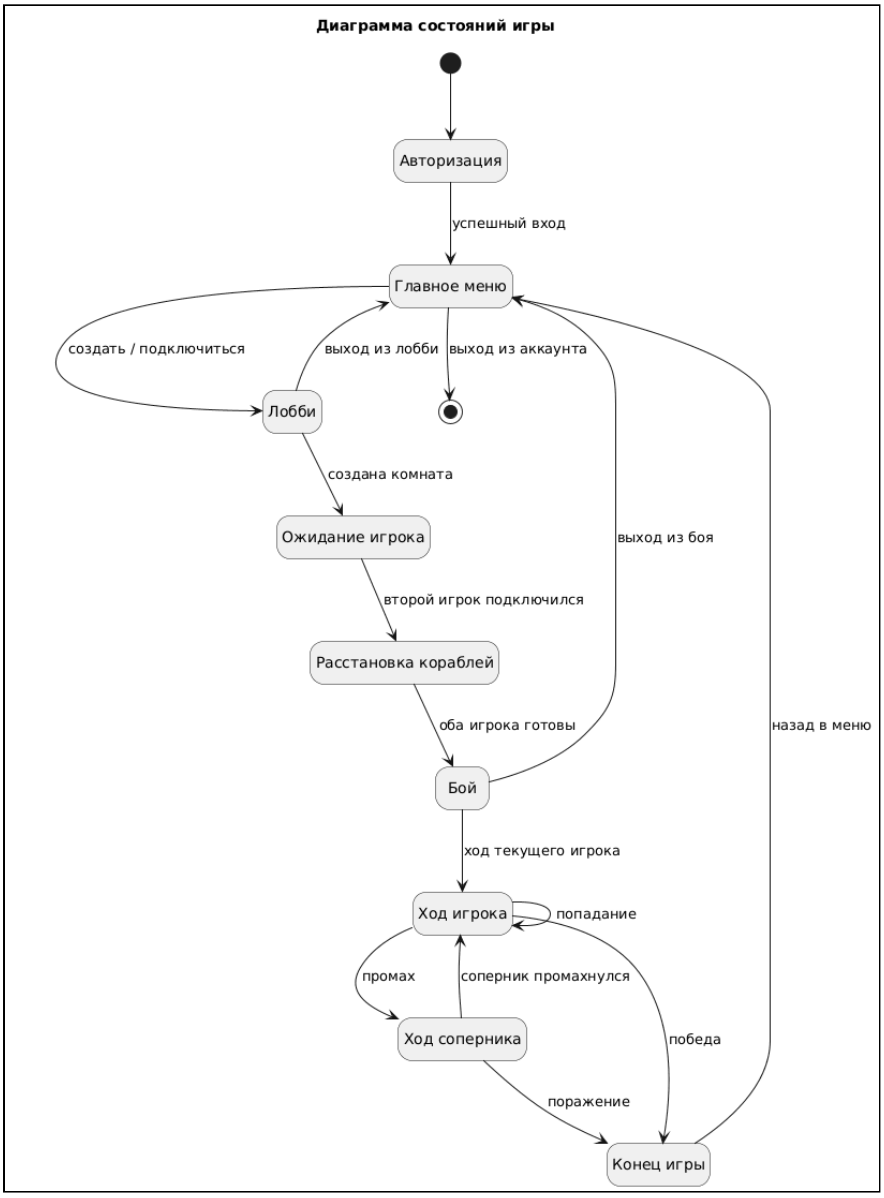

Рисунок 2.3 – Диаграмма состояний игры

2.2 Профиль пользователя

После входа пользователь может настроить профиль: указать имя и выбрать аватар. Выбранный аватар сохраняется вместе с профилем и далее используется в лобби и на экране боя. Для аватаров используется перечисление *AvatarChoice*, а выбранное значение сохраняется в базе данных как строковый идентификатор.

На рисунке 2.4 представлен экран профиля пользователя.

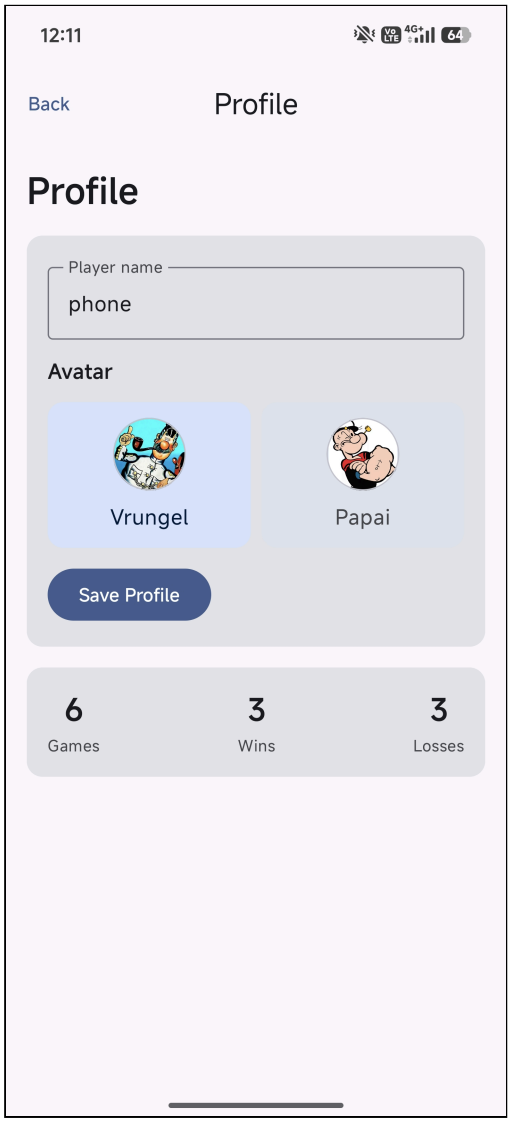

Рисунок 2.4 – Экран профиля пользователя

2.3 Создание лобби и расстановка кораблей

Игрок может создать новое лобби или подключиться к существующему по коду. После подключения оба игрока переходят на экран лобби. На этом экране отображается код комнаты, список игроков, состояние готовности и поле для расстановки кораблей.

Расстановка кораблей выполняется на поле 10 на 10 клеток. Пользователь выбирает размер корабля и его ориентацию. При попытке поставить корабль в неправильное место приложение показывает сообщение об ошибке. После размещения всех кораблей игрок может отметить себя готовым.

На рисунке 2.5 представлен экран лобби.

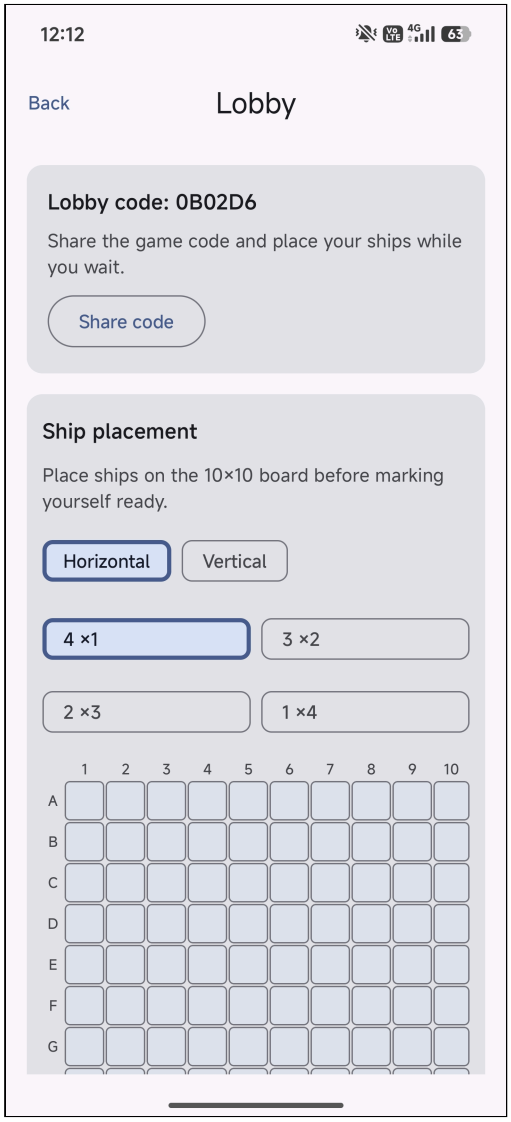

Рисунок 2.5 – Экран лобби и расстановки кораблей

2.4 Игровой процесс

После готовности двух игроков начинается бой. В игре отображаются два поля: собственное поле игрока и поле соперника. Игрок выбирает клетку на поле соперника. Если попадание успешно, игрок продолжает ход. При промахе ход передается сопернику.

При уничтожении корабля клетки вокруг него автоматически отмечаются как недоступные для дальнейших выстрелов, так как по правилам корабли не могут касаться друг друга. Победителем становится игрок, уничтоживший все корабли соперника.

На рисунке 2.6 представлен экран сражения.

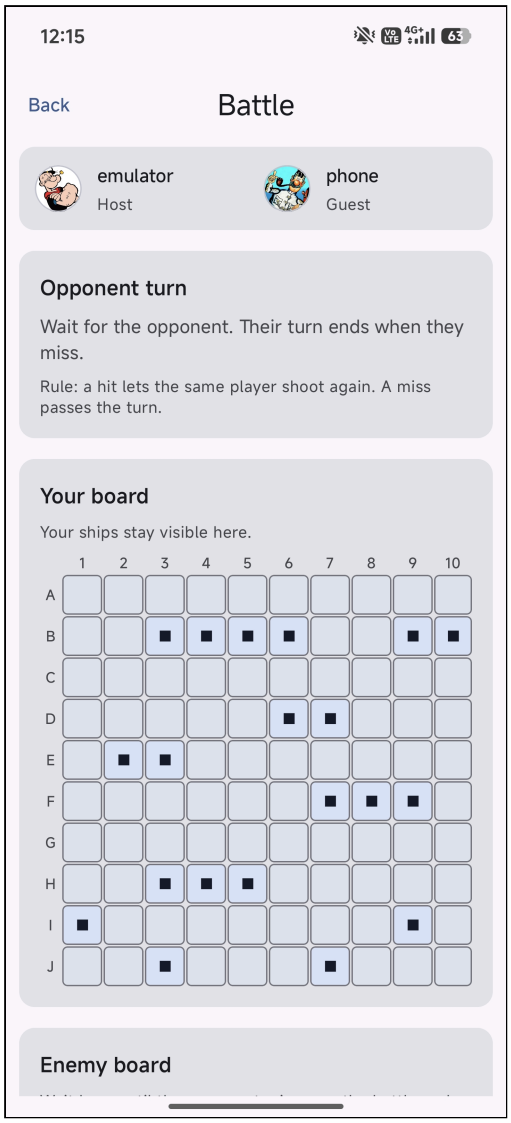

Рисунок 2.6 – Экран сражения

2.5 Навигация и выход из игры

В приложении реализована обработка кнопки возврата и системного жеста назад. В лобби выход выполняется сразу, так как игрок еще не начал бой. На экране сражения перед выходом показывается диалог подтверждения, чтобы пользователь случайно не покинул матч.

При выходе игрока из лобби или боя локальное состояние очищается, а запись игры удаляется из Firestore. Это позволяет второму игроку корректно получить информацию о завершении текущей партии или отключении соперника.

ВЫВОД

В ходе лабораторной работы было разработано мобильное приложение «Морской бой» для Android. В приложении реализованы авторизация пользователей, настройка профиля, создание и подключение к игровому лобби, расстановка кораблей, сетевой обмен состоянием через Firebase Firestore и проведение боя между двумя игроками. Также были построены диаграммы последовательности, развертывания и состояний, описывающие основные части работы приложения.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Обработка выстрела игрока в GameEngine

```
  fun applyHostShot(state: GameState, cellIndex: Int): GameState {
      require(state.status == GameStatus.HOST_TURN) { "Host turn
expected" }
      require(cellIndex !in state.hostShotsMade) { "Cell already
targeted" }

      var updatedShotsMade = state.hostShotsMade + cellIndex
      var updatedGuestShotsReceived = state.guestShotsReceived +
cellIndex

      val hitShip = state.guestShips.find { cellIndex in it.cells }
      val isHit = hitShip != null

      val hostWon =
allShipCells(state.guestShips).all(updatedGuestShotsReceived::contains)

      return state.copy(
          hostShotsMade = updatedShotsMade,
          guestShotsReceived = updatedGuestShotsReceived,
          status = if (hostWon) GameStatus.FINISHED else state.status,
          winnerUid = if (hostWon) state.hostUid else null,
      )
  }
```

Листинг А.2 – Создание игры в GameRepository

```
  suspend fun createGame(hostUid: String, hostProfile: Profile):
String {
      val gameId = buildGameId()
      firestore.collection(FirebaseRefs.Games)
          .document(gameId)
          .set(
              mapOf(
                  "gameId" to gameId,
                  "hostUid" to hostUid,
                  "hostProfile" to profileToMap(hostProfile),
                  "status" to GameStatus.WAITING_FOR_GUEST.name,
              )
          )
          .await()
      return gameId
  }
```